Ecuaciones diferenciales ordinarias II: método de Runge–Kutta de cuarto orden y `solve_ivp`

Material de teoría

Índice

1. Introducción 3

2. Problema de valor inicial 3

3. Motivación: de Euler a Runge–Kutta 3

4. Método de Runge–Kutta de cuarto orden 4
- 4.1. Primera pendiente . . . 4
- 4.2. Segunda pendiente . . . 4
- 4.3. Tercera pendiente . . . 4
- 4.4. Cuarta pendiente . . . 4
- 4.5. Fórmula de actualización . . . 4

5. Interpretación de las cuatro pendientes 5

6. Algoritmo RK4 5

7. Orden y error numérico 5

8. Implementación de RK4 en Python 6

9. Ejemplo: decaimiento exponencial 7

10. Estudio experimental del tamaño de paso 7

11. Resolución de EDO mediante SciPy 8

12. Puntos de evaluación: `t_eval` 8

13. Métodos de integración 9

14. Tolerancias: `rtol` y `atol` 9

15. Ejemplo físico: caída con resistencia lineal 9

16. Sistemas de ecuaciones diferenciales 10

17. Ejemplo: oscilador armónico 11

18. Consideraciones sobre el trabajo numérico **11**

19. Ejercicios propuestos **12**
19.1. Ejercicio 1: ecuación logística 12
19.2. Ejercicio 2: oscilador armónico 12

20. Resumen **12**

1. Introducción

Una ecuación diferencial ordinaria (EDO) relaciona una función desconocida con una o más de sus derivadas. En esta unidad estudiaremos métodos numéricos para resolver problemas de valor inicial de la forma

$$\frac{dy}{dt} = f(t, y), \qquad y(t_0) = y_0.$$

En las clases anteriores se estudiaron métodos como Euler y Euler mejorado. Estos métodos permiten aproximar una solución, pero su precisión puede ser limitada cuando se utilizan pasos grandes. El método de Runge–Kutta de cuarto orden, conocido como RK4, permite obtener una aproximación mucho más precisa sin requerir una fórmula analítica para la solución.

También estudiaremos la función `solve_ivp` de la biblioteca SciPy, que permite resolver problemas de valor inicial utilizando diferentes algoritmos numéricos y estrategias de control del paso.

El objetivo es comprender tanto el método numérico como su implementación computacional.

2. Problema de valor inicial

Consideremos una EDO de primer orden

$$\frac{dy}{dt} = f(t, y)$$

junto con una condición inicial

$$y(t_0) = y_0.$$

El objetivo es aproximar la función $y(t)$ en un intervalo

$$t_0 \leq t \leq t_f.$$

Dividimos el intervalo en puntos

$$t_0, t_1, t_2, \ldots, t_N$$

separados por un tamaño de paso h:

$$t_{n+1} = t_n + h.$$

La solución numérica estará dada por los valores

$$y_0, y_1, \ldots, y_N,$$

donde y_n aproxima la solución exacta en t_n.

3. Motivación: de Euler a Runge–Kutta

El método de Euler utiliza la pendiente de la función al comienzo de cada intervalo:

$$y_{n+1} = y_n + hf(t_n, y_n).$$

La idea geométrica es aproximar la solución mediante una recta cuya pendiente es la pendiente inicial.

El inconveniente es que la pendiente puede cambiar considerablemente dentro del intervalo. Una posibilidad para mejorar la aproximación consiste en evaluar la pendiente en varios puntos del intervalo y combinar dichas evaluaciones.

Los métodos de Runge–Kutta siguen precisamente esta idea.

4. Método de Runge–Kutta de cuarto orden

Para la ecuación

$$\frac{dy}{dt} = f(t, y),$$

el método RK4 utiliza cuatro estimaciones de la pendiente.

4.1. Primera pendiente

La primera evaluación se realiza al comienzo del intervalo:

$$k_1 = f(t_n, y_n).$$

Esta es la pendiente conocida directamente en el punto (t_n, y_n).

4.2. Segunda pendiente

Utilizamos k_1 para realizar una estimación en el punto medio:

$$k_2 = f\left(t_n + \frac{h}{2}, y_n + \frac{h}{2}k_1\right).$$

Aquí se ha avanzado temporalmente una distancia $h/2$ y se ha estimado el valor de y utilizando la pendiente k_1.

4.3. Tercera pendiente

Se realiza nuevamente una evaluación en el punto medio, pero utilizando k_2:

$$k_3 = f\left(t_n + \frac{h}{2}, y_n + \frac{h}{2}k_2\right).$$

La segunda y tercera evaluaciones proporcionan información más representativa de lo que ocurre en la región central del intervalo.

4.4. Cuarta pendiente

Finalmente se estima la pendiente al final del intervalo:

$$k_4 = f(t_n + h, y_n + hk_3).$$

4.5. Fórmula de actualización

Las cuatro pendientes se combinan mediante

$$\boxed{y_{n+1} = y_n + \frac{h}{6}\left(k_1 + 2k_2 + 2k_3 + k_4\right)}$$

Los pesos utilizados son

$$1, \qquad 2, \qquad 2, \qquad 1.$$

Por esta razón, las dos evaluaciones realizadas en el punto medio tienen mayor peso que las evaluaciones de los extremos.

5. Interpretación de las cuatro pendientes

El esquema puede resumirse de la siguiente manera:

Pendiente	Ubicación	Interpretación
k_1	Inicio	Pendiente inicial
k_2	Punto medio	Primera estimación intermedia
k_3	Punto medio	Segunda estimación intermedia
k_4	Final	Pendiente final estimada

La principal ventaja conceptual de RK4 es que no intenta describir todo el intervalo utilizando solamente una pendiente.

6. Algoritmo RK4

Partiendo del punto (t_n, y_n):

1. Calcular k_1.
2. Calcular k_2 utilizando k_1.
3. Calcular k_3 utilizando k_2.
4. Calcular k_4 utilizando k_3.
5. Calcular y_{n+1} mediante la combinación ponderada.
6. Avanzar hasta el siguiente intervalo.

En pseudocódigo:

```
k1 = f(tn, yn)

k2 = f(tn + h/2,
       yn + h*k1/2)

k3 = f(tn + h/2,
       yn + h*k2/2)

k4 = f(tn + h,
       yn + h*k3)

yn1 = yn + h*(k1 + 2*k2 + 2*k3 + k4)/6
```

7. Orden y error numérico

Un método numérico tiene un orden asociado con la rapidez con la que disminuye su error al reducir el tamaño de paso.

Para RK4:

$$\boxed{\text{error local} = O(h^5)}$$

y

$$\boxed{\text{error global} = O(h^4)}.$$

El error global se refiere al error acumulado en la aproximación sobre el intervalo de integración.

Si el comportamiento asintótico domina, al reducir el paso a la mitad se espera aproximadamente

$$E(h/2) \approx \frac{E(h)}{2^4} = \frac{E(h)}{16}.$$

Esto no significa que toda ejecución produzca exactamente ese factor. Para pasos suficientemente pequeños pueden comenzar a ser relevantes otros errores, especialmente los asociados con la representación de números en punto flotante.

8. Implementación de RK4 en Python

Una implementación básica para una EDO escalar es:

```
import numpy as np

def rk4(f, t0, tf, y0, h):
    t = np.arange(t0, tf + h, h)
    y = np.zeros(len(t), dtype=float)

    y[0] = y0

    for n in range(len(t) - 1):

        k1 = f(t[n], y[n])

        k2 = f(
            t[n] + h/2,
            y[n] + h*k1/2
        )

        k3 = f(
            t[n] + h/2,
            y[n] + h*k2/2
        )

        k4 = f(
            t[n] + h,
            y[n] + h*k3
        )

        y[n+1] = y[n] + h/6 * (
            k1 + 2*k2 + 2*k3 + k4
        )

    return t, y
```

La función recibe:

- `f`: la función $f(t, y)$.
- `t0`: tiempo inicial.
- `tf`: tiempo final.
- `y0`: condición inicial.
- `h`: tamaño del paso.

Devuelve los vectores de tiempos y de aproximaciones numéricas.

9. Ejemplo: decaimiento exponencial

Consideremos

$$\frac{dy}{dt} = -2y, \qquad y(0) = 1.$$

Esta ecuación tiene solución exacta

$$\boxed{y(t) = e^{-2t}}.$$

La función que representa la EDO es:

```
def f(t, y):
    return -2*y
```

Podemos aplicar RK4 con

$$h = 0{,}1$$

mediante:

```
t, y_rk4 = rk4(
    f,
    0,
    5,
    1,
    0.1
)
```

La solución exacta se calcula como:

```
y_exacta = np.exp(-2*t)
```

El error absoluto en cada punto es

$$E_n = \left| y_{\text{exacta}}(t_n) - y_{\text{RK4}}(t_n) \right|.$$

En Python:

```
error = np.abs(y_exacta - y_rk4)

print("Error maximo:", np.max(error))
```

10. Estudio experimental del tamaño de paso

Para estudiar el orden de RK4 podemos utilizar diferentes valores de h:

$$h = 0{,}5, \qquad h = 0{,}1, \qquad h = 0{,}05, \qquad h = 0{,}01.$$

Para cada valor calculamos

$$E_{\text{máx}} = \max_n \left| y_{\text{exacta}}(t_n) - y_{\text{RK4}}(t_n) \right|.$$

Si RK4 se encuentra en su régimen asintótico, esperamos observar aproximadamente

$$E_{\text{máx}} \propto h^4.$$

Una forma práctica de analizarlo consiste en comparar los errores obtenidos al reducir el paso.

11. Resolución de EDO mediante SciPy

La biblioteca SciPy incluye métodos numéricos para resolver problemas de valor inicial.
La función que utilizaremos es

```
from scipy.integrate import solve_ivp
```

La estructura básica es:

```
sol = solve_ivp(
    fun,
    t_span,
    y0
)
```

donde:

- `fun` representa la función $f(t, y)$.
- `t_span` especifica el intervalo de integración.
- `y0` contiene la condición inicial.

Por ejemplo:

```
def f(t, y):
    return -2*y

sol = solve_ivp(
    f,
    (0, 5),
    [1]
)
```

12. Puntos de evaluación: `t_eval`

Podemos indicar los tiempos en los que queremos conocer la solución utilizando `t_eval`.
Por ejemplo:

```
t_eval = np.linspace(0, 5, 100)

sol = solve_ivp(
    f,
    (0, 5),
    [1],
    t_eval=t_eval
)
```

Los tiempos calculados se encuentran en:

```
sol.t
```

y las soluciones en:

```
sol.y
```

Para una única variable, podemos acceder a la primera fila mediante:

```
sol.y[0]
```

13. Métodos de integración

`solve_ivp` permite seleccionar diferentes métodos numéricos mediante el argumento `method`.

Uno de los métodos disponibles es:

```
method="RK45"
```

RK45 utiliza un esquema Runge–Kutta con estimación embebida del error y control adaptativo del tamaño de paso.

Es importante diferenciarlo de nuestra implementación manual de RK4:

Característica	RK4 manual	RK45 en `solve_ivp`
Paso	Fijo	Adaptativo
Implementación	Explícita	Biblioteca
Control de error	Estudiado manualmente	Incorporado
Uso principal	Comprensión del algoritmo	Resolución general

14. Tolerancias: `rtol` y `atol`

En `solve_ivp` pueden especificarse tolerancias mediante

```
rtol
atol
```

`rtol` corresponde a la tolerancia relativa y `atol` a la tolerancia absoluta.

Conceptualmente, el control del error utiliza una escala relacionada con

$$\texttt{atol} + \texttt{rtol}|y|.$$

Modificar las tolerancias puede modificar el tamaño de los pasos internos utilizados por el algoritmo.

Una tolerancia más estricta no debe interpretarse automáticamente como una solución "más verdadera". Siempre conviene analizar el modelo, el método y la precisión necesaria para el problema.

15. Ejemplo físico: caída con resistencia lineal

Consideremos una masa que cae bajo la acción de la gravedad y una fuerza de resistencia proporcional a la velocidad:

$$m\frac{dv}{dt} = mg - cv.$$

Despejando:

$$\boxed{\frac{dv}{dt} = g - \frac{c}{m}v}.$$

Supongamos

$$m = 2\,\text{kg}, \qquad c = 0{,}5\,\text{kg/s}, \qquad g = 9{,}81\,\text{m/s}^2.$$

La condición inicial es

$$v(0) = 0.$$

La velocidad terminal se obtiene cuando la aceleración es cero:

$$0 = g - \frac{c}{m}v_T.$$

Por tanto,

$$\boxed{v_T = \frac{mg}{c}}$$

y para los valores utilizados:

$$v_T = 39{,}24\,\mathrm{m/s}.$$

La implementación es:

```
import numpy as np
from scipy.integrate import solve_ivp

m = 2.0
c = 0.5
g = 9.81

def dvdt(t, v):
    return g - (c/m)*v

t_eval = np.linspace(0, 20, 200)

sol = solve_ivp(
    dvdt,
    (0, 20),
    [0],
    t_eval=t_eval
)
```

La solución numérica permite observar cómo la velocidad se aproxima progresivamente a la velocidad terminal.

16. Sistemas de ecuaciones diferenciales

Muchos problemas físicos se expresan naturalmente como sistemas de EDO.
Una ecuación diferencial de segundo orden

$$\frac{d^2x}{dt^2} + \omega^2 x = 0$$

puede transformarse en un sistema de primer orden.
Definimos

$$v = \frac{dx}{dt}.$$

Entonces:

$$\frac{dx}{dt} = v$$

y

$$\frac{dv}{dt} = -\omega^2 x.$$

Por tanto:

$$\boxed{\begin{aligned} \frac{dx}{dt} &= v, \\ \frac{dv}{dt} &= -\omega^2 x. \end{aligned}}$$

Definimos el vector de estado

$$\mathbf{y} = \begin{pmatrix} x \\ v \end{pmatrix}.$$

El sistema puede escribirse como

$$\frac{d\mathbf{y}}{dt} = \mathbf{f}(t, \mathbf{y}).$$

17. Ejemplo: oscilador armónico

Para

$$\omega = 2, \qquad x(0) = 1, \qquad v(0) = 0,$$

podemos utilizar:

```
omega = 2.0

def oscilador(t, y):
    x, v = y

    dxdt = v
    dvdt = -(omega**2)*x

    return [dxdt, dvdt]
```

La solución se obtiene con:

```
t_eval = np.linspace(0, 10, 500)

sol = solve_ivp(
    oscilador,
    (0, 10),
    [1, 0],
    t_eval=t_eval
)
```

La primera fila de `sol.y` corresponde a $x(t)$ y la segunda a $v(t)$.
Para estas condiciones iniciales, la solución analítica de la posición es

$$\boxed{x(t) = \cos(\omega t)}.$$

Por tanto, podemos comparar la solución numérica con la solución exacta.

18. Consideraciones sobre el trabajo numérico

Una solución numérica debe analizarse, no solamente calcularse. Algunas buenas prácticas son:

- Verificar las condiciones iniciales.
- Revisar las unidades físicas.
- Comparar con una solución exacta cuando esté disponible.
- Estudiar la influencia del tamaño de paso.
- Analizar el error máximo u otra medida de error apropiada.
- Graficar los resultados.
- Documentar el método y los parámetros utilizados.
- Distinguir entre error numérico y error asociado al modelo físico.

19. Ejercicios propuestos

19.1. Ejercicio 1: ecuación logística

Resolver

$$\frac{dy}{dt} = y(1-y), \qquad y(0) = 0{,}1,$$

en el intervalo

$$0 \le t \le 10.$$

Se solicita:

1. Implementar el método RK4.
2. Resolver la ecuación mediante `solve_ivp`.
3. Utilizar los mismos puntos de evaluación para comparar ambos resultados.
4. Graficar las dos soluciones.
5. Calcular el error máximo entre las soluciones numéricas.

19.2. Ejercicio 2: oscilador armónico

Resolver

$$\frac{d^2x}{dt^2} + \omega^2 x = 0$$

con

$$x(0) = 1, \qquad v(0) = 0, \qquad \omega = 2.$$

Se solicita:

1. Transformar la ecuación de segundo orden en un sistema de primer orden.
2. Resolver el sistema mediante `solve_ivp`.
3. Graficar $x(t)$ y $v(t)$.
4. Comparar $x(t)$ con la solución analítica $\cos(\omega t)$.
5. Calcular el error máximo en la posición.

20. Resumen

Las ideas fundamentales de esta unidad son:

1. RK4 utiliza cuatro evaluaciones de la pendiente en cada paso.
2. La actualización es
$$y_{n+1} = y_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4).$$
3. RK4 tiene error global de orden
$$O(h^4).$$
4. La implementación manual permite comprender el funcionamiento del algoritmo.
5. `solve_ivp` permite resolver problemas de valor inicial utilizando diferentes métodos.

6. Métodos como RK45 pueden adaptar automáticamente el tamaño del paso.

7. `solve_ivp` también permite resolver sistemas de ecuaciones diferenciales.

8. La calidad de una solución numérica debe analizarse mediante error, precisión y comportamiento físico o matemático.